

LAPORAN PRAKTIKUM
WORKSHOP SISTEM INFORMASI WEB SERVER
(TIF330805)
SEMESTER III



Acara 14

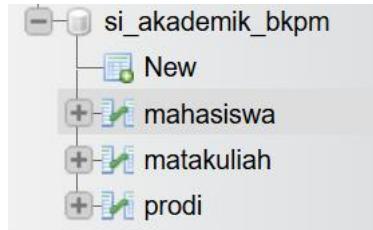
Nama:
Wildan Ramadhani Akbar

NIM:
E41250425

PROGRAM STUDI DILUAR KAMPUS UTAMA (PSDKU)
TEKNIK INFORMATIKA KAB. NGANJUK
JURUSAN TEKNOLOGI INFORMASI
POLITEKNIK NEGERI JEMBER
TAHUN 2026

1. Tugas

- Membuat db si_akademik



- Menghubungkan aplikasi ke db menggunakan pdo dan memastikan repository menggunakan prepare statement

```
8 references
private PDO $db;

1 reference | 0 overrides
public function __construct(Database $database)
{
    $this->db = $database->getConnection();
}

1 reference | 0 overrides
public function all(string $search = ''): array
{
    $statement = $this->db->prepare(
        'SELECT mahasiswa.id, mahasiswa.nim, mahasiswa.nama,
        FROM mahasiswa
        LEFT JOIN prodi ON prodi.id = mahasiswa.prodi_id
        WHERE mahasiswa.nama LIKE :name OR mahasiswa.nim LI
        ORDER BY mahasiswa.id'
    );
    $term = '%' . $search . '%';
    $statement->execute(['name' => $term, 'nim' => $term]);
    return $statement->fetchAll();
}
```

- Method all, find, create, update, delete, existByNim

```
1 reference | 0 overrides
public function all(string $search = ''): array
{
    $statement = $this->db->prepare(
        'SELECT mahasiswa.id, mahasiswa.nim, mahasiswa.nama, mahasiswa.angkatan,
        FROM mahasiswa
        LEFT JOIN prodi ON prodi.id = mahasiswa.prodi_id
        WHERE mahasiswa.nama LIKE :name OR mahasiswa.nim LIKE :nim
        ORDER BY mahasiswa.id'
    );
    $term = '%' . $search . '%';
    $statement->execute(['name' => $term, 'nim' => $term]);

    return $statement->fetchAll();
}
```

```
1 reference | 0 overrides
public function find(int $id): ?array
{
    $statement = $this->db->prepare('SELECT * FROM mahasiswa WHERE id = :id');
    $statement->execute(['id' => $id]);

    return $statement->fetch() ?: null;
}
```

```
1 reference | 0 overrides
public function create(array $data): bool
{
    $statement = $this->db->prepare(
        'INSERT INTO mahasiswa (nim, nama, email, prodi_id, angkatan, status)
        VALUES (:nim, :nama, :email, :prodi_id, :angkatan, :status)'
    );

    return $statement->execute($data);
}

1 reference | 0 overrides
public function update(int $id, array $data): bool
{
    $statement = $this->db->prepare(
        'UPDATE mahasiswa
        SET nim = :nim, nama = :nama, email = :email, prodi_id = :prodi_id,
        angkatan = :angkatan, status = :status
        WHERE id = :id'
    );
    $data['id'] = $id;

    return $statement->execute($data);
}
```

1 reference | 0 overrides

```
public function delete(int $id): bool
{
    $statement = $this->db->prepare('DELETE FROM mahasiswa WHERE id = :id');

    return $statement->execute(['id' => $id]);
}
```

2 references | 0 overrides

```
public function existsByNim(string $nim, ?int $excludeId = null): bool
{
    $sql = 'SELECT 1 FROM mahasiswa WHERE nim = :nim';
    $parameters = ['nim' => $nim];

    if ($excludeId !== null) {
        $sql .= ' AND id <> :id';
        $parameters['id'] = $excludeId;
    }

    $statement = $this->db->prepare($sql);
    $statement->execute($parameters);

    return $statement->fetchColumn() !== false;
}
```

- MahasiswaService validasi data dan memanggil repository

```

2 references
private function validatedData(array $input): ?array
{
    $nim = $input['nim'] ?? '';
    $nama = $input['nama'] ?? '';
    $email = $input['email'] ?? '';
    $status = $input['status'] ?? '';
    $data = [
        'nim' => is_scalar($nim) ? trim((string) $nim) : '',
        'nama' => is_scalar($nama) ? trim((string) $nama) : '',
        'email' => is_scalar($email) ? trim((string) $email) : '',
        'prodi_id' => filter_var($input['prodi_id'] ?? null, FILTER_VALIDATE_INT) ?: 0,
        'angkatan' => filter_var($input['angkatan'] ?? null, FILTER_VALIDATE_INT) ?: 0,
        'status' => is_scalar($status) ? (string) $status : '',
    ];

    if (
        !ctype_digit($data['nim']) || $data['nama'] === '' ||
        !filter_var($data['email'], FILTER_VALIDATE_EMAIL) ||
        $data['prodi_id'] < 1 || $data['angkatan'] < 1 ||
        !in_array($data['status'], ['aktif', 'cuti', 'lulus'], true)
    ) {
        return null;
    }
}

```

```

1 reference | 0 overrides
public function __construct(
    private MahasiswaRepository $mahasiswaRepository,
    private ProdiRepository $prodiRepository
) {
}

```

- Try catch pada proses crud yang berpotensi error

```

$search = trim($_GET['q'] ?? '');
try {
    $mahasiswaList = $this->mahasiswaRepository->all($search);
} catch (Throwable $exception) {
    Logger::error($exception);
    $mahasiswaList = [];
    $_SESSION['flash_error'] = 'Data gagal dimuat.';
}

```

- Logging supaya Ketika ada error aka nada log yang dikirim di storage/log

```
8 references | 0 overrides
public static function error(Throwable $exception): void
{
    $entry = sprintf(
        "[%s] %s: %s in %s:%d\n%s\n\n",
        date('Y-m-d H:i:s'),
        get_class($exception),
        $exception->getMessage(),
        $exception->getFile(),
        $exception->getLine(),
        $exception->getTraceAsString()
    );
    $directory = dirname(__DIR__, 2) . '/storage/logs';
```

- Flash message

```
['flash_success'] = 'Data mahasiswa berhasil dihapus.';

['flash_error'] = 'Data gagal dihapus.';
```